BÁO CÁO LAB-01

Cấu hình Kernel và Khởi động Hệ thống Nhúng (ARMv7)

Môn học: Embedded Linux Systems

DANH SÁCH THÀNH VIÊN NHÓM

Họ và tên	MSSV
Lê Tiến Thành	SE201799
Lê Tấn Lộc	SE201412
Nguyễn Quang Huy Đạt	SE201415
Trần Đình Khôi Nguyên	SE200558
Trần Trương Tuấn Vũ	SE196467

Tháng 9–10 năm 2026

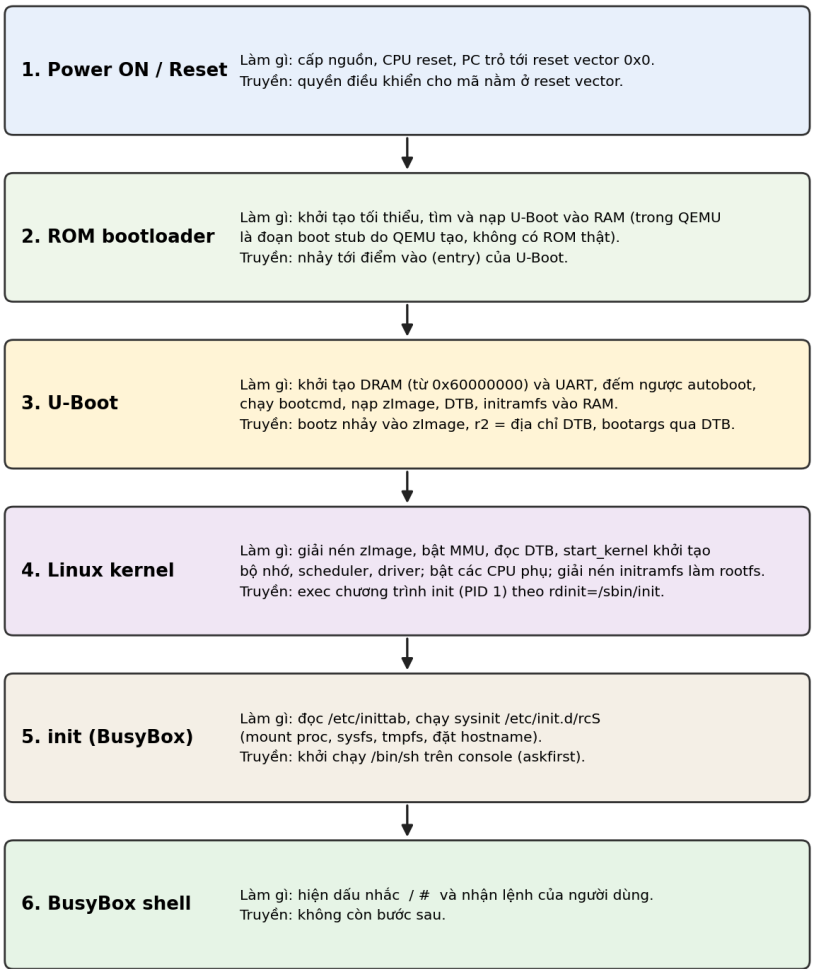
Mục 1. Thông tin sinh viên

Bảng 1. Thông tin sinh viên

Mục	Nội dung
Đại diện nhóm	Lê Tiến Thành
MSSV đại diện	SE201799
Thành viên	05 sinh viên (danh sách họ tên và MSSV tại trang bìa)
Ngày thực hiện	13/09/2026 (build kernel, BusyBox, initramfs, boot) và 15/09/2026 (kiểm tra lại cấu hình kernel)
Môi trường	QEMU vexpress-a9 (Cortex-A9), Linux 5.15.0, BusyBox 1.35.0, U-Boot 2022.04, toolchain arm-linux-gnueabihf

Mục 2. Boot sequence diagram

Sơ đồ dưới đây vẽ chuỗi khởi động của hệ thống trong bài lab, từ lúc cấp nguồn đến khi có dấu nhắc của BusyBox shell. Mỗi khối ghi thành phần đang chạy, việc chính của nó và thứ nó truyền cho bước sau. Trong QEMU không có nguồn điện và ROM thật, nên bước 1 và 2 được QEMU mô phỏng.



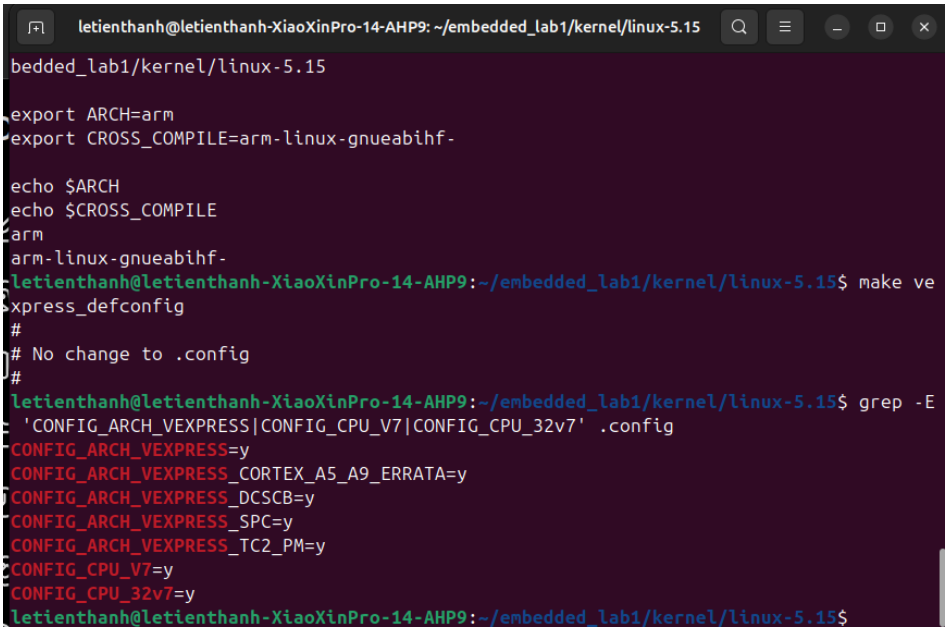
Hình 1. Chuỗi khởi động từ Power ON đến BusyBox shell

Hai điểm trong sơ đồ em đã kiểm chứng bằng kết quả chạy. Thứ nhất, PID 1 là `init` và shell là `-/bin/sh` (Hình 19), đúng với cách BusyBox init chạy `askfirst` trong `/etc/inittab`. Thứ hai, `mount` liệt kê `proc`, `sysfs` và `tmpfs` trên `/tmp` (Hình 18), là những gì `/etc/init.d/rcS` mount. Các bước ở U-Boot dựa trên cấu hình trong đề (địa chỉ nạp kernel, initramfs và DTB), kết quả thực tế nằm ở các hình của phần B.

Mục 3. Log thực hành

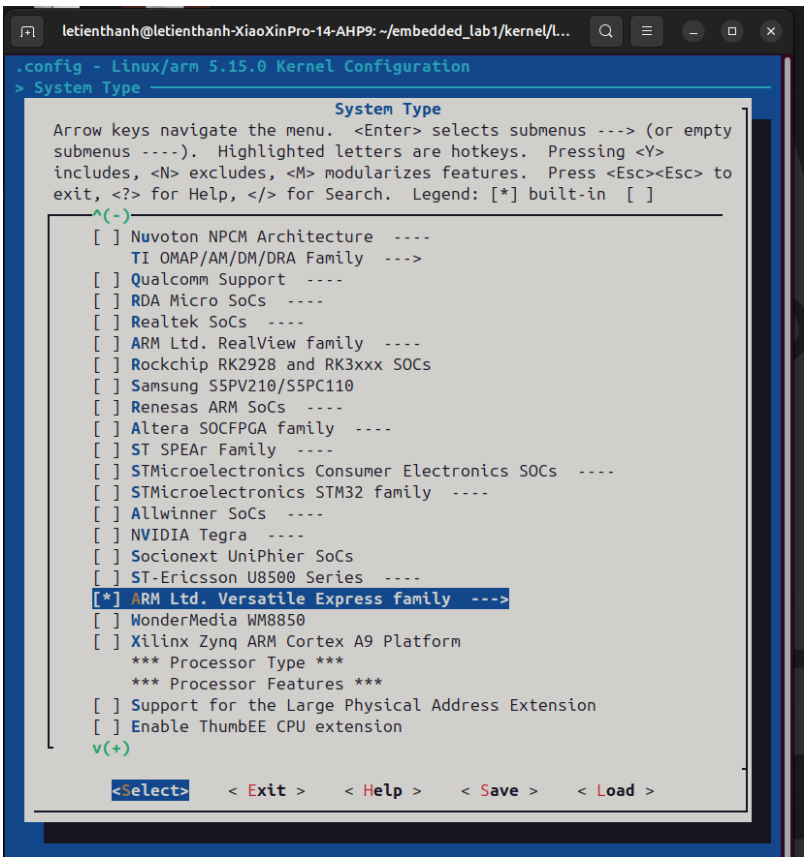
Phần A. Build kernel

Em đặt `ARCH=arm` và `CROSS_COMPILE=arm-linux-gnueabihf-` rồi dùng `vexpress_defconfig` như mục A.2 của đề (đề ghi `versatile_defconfig` ở phần yêu cầu báo cáo, em dùng `vexpress_defconfig` vì đó là cấu hình đúng cho vexpress-a9). Sau khi nạp cấu hình, lệnh `grep` cho thấy `CONFIG_ARCH_VEXPRESS=y`, `CONFIG_CPU_V7=y` và `CONFIG_CPU_32v7=y`.



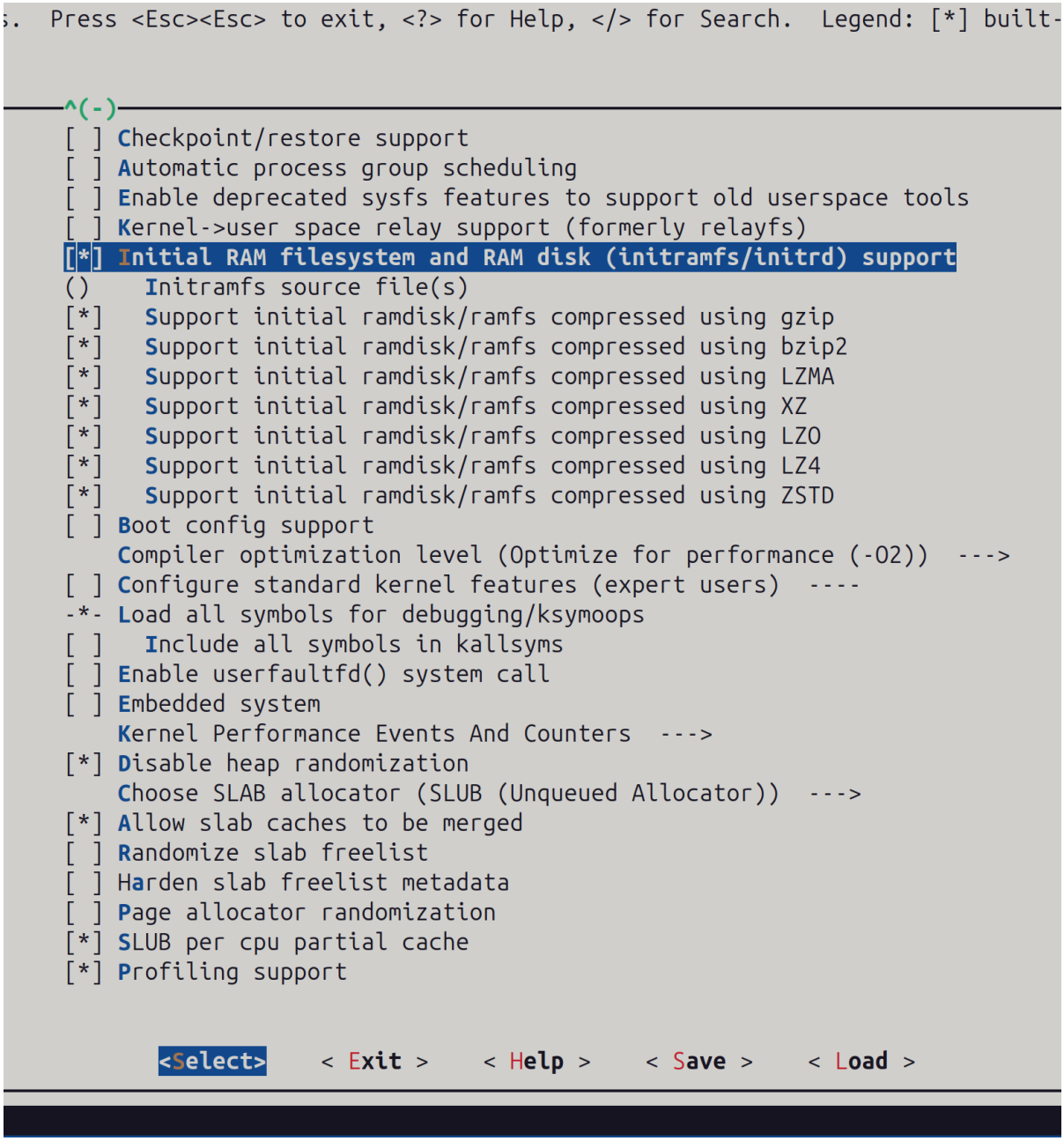
Hình 2. make vexpress_defconfig và kiểm tra CONFIG_ARCH_VEXPRESS, CONFIG_CPU_V7

Trong `menuconfig` (bản 5.15 hiển thị là Linux/arm 5.15.0), em xem các mục sau. Mục "Platform selection" trong đề nằm dưới tên System Type: tùy chọn `ARM Ltd. Versatile Express family` đã được bật, còn `Support for the Large Physical Address Extension` (LPAE) đang tắt, vì máy chỉ có 512 MB RAM.



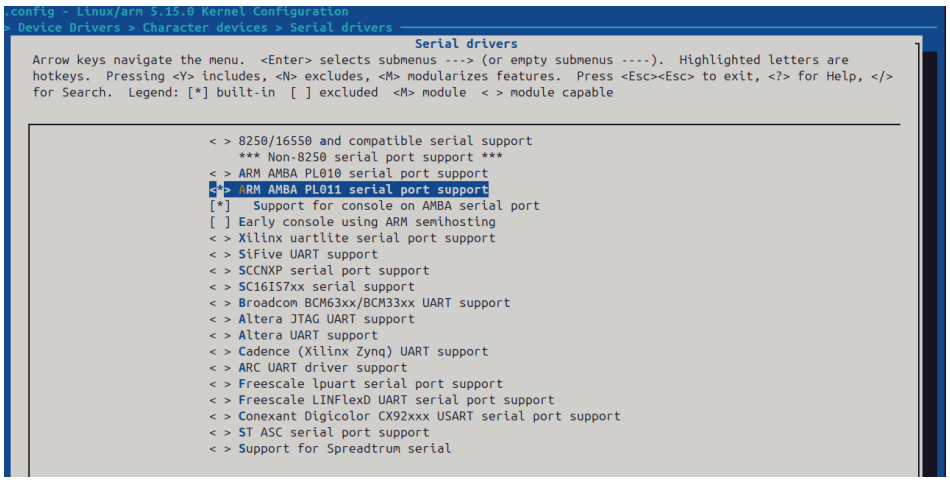
Hình 3. System Type: Versatile Express family bật, LPAE tắt

Để bật initramfs em vào General setup, rồi Initial RAM filesystem and RAM disk (initramfs/initrd) support và đánh dấu [*]. Đường dẫn menu đầy đủ là: General setup, Initial RAM filesystem and RAM disk (initramfs/initrd) support.



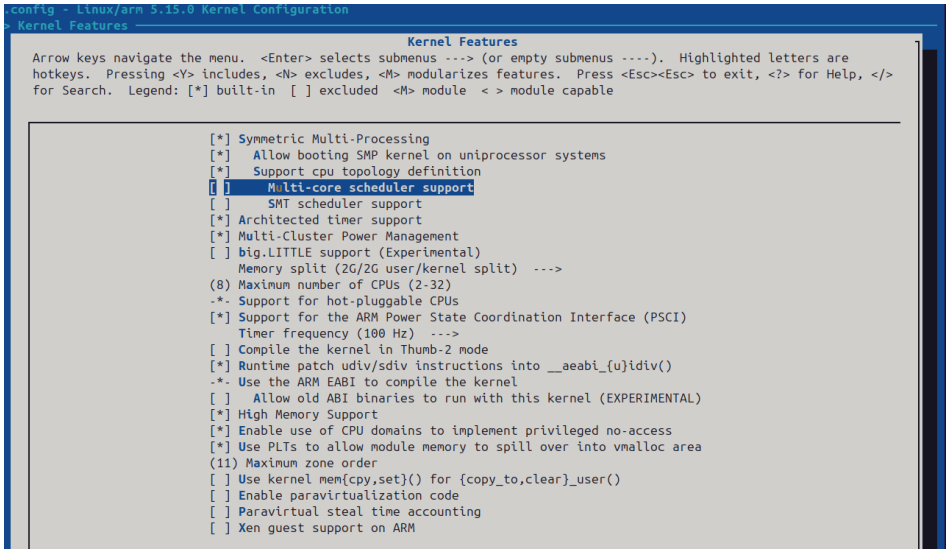
Hình 4. General setup, Initial RAM filesystem and RAM disk (initramfs/initrd) support được bật [*] (chụp trong menuconfig ARM)

Console của vexpress-a9 dùng UART PL011. Trong Device Drivers, Character devices, Serial drivers, `ARM AMBA PL011 serial port support` được bật dạng built-in <*> và `Support for console on AMBA serial port` được bật [*]. Hai tùy chọn này cho `console=ttyAMA0`.



Hình 5. Serial drivers: PL011 và console trên AMBA serial port

Ở bước tùy chọn em bật SMP. Trong Kernel Features, `Symmetric Multi-Processing` được đánh dấu [*], số CPU tối đa là 8, có hỗ trợ CPU hot-plug và PSCI, tần số timer 100 Hz. Mục Memory split đang là 2G/2G (user/kernel).



Hình 6. Kernel Features: SMP bật, Memory split 2G/2G



Hình 7. Kernel Features, Memory split: danh sách các lựa chọn của mục Memory split

Khi chạy `make -j$(nproc) zImage dtbs modules`, kernel được build xong lúc 10:02:57 ngày 13/09/2026 (thời điểm này có trong dòng `uname -a` ở Hình 18). Kết quả build như sau.

```
  LD      .tmp_vmlinux.kallsyms1
arm-linux-gnueabihf-ld: warning: arch/arm/lib/ucmpdi2.o: missing .note.GNU-stack section implies executable stack
arm-linux-gnueabihf-ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
  KSYMS   .tmp_vmlinux.kallsyms1.S
  AS      .tmp_vmlinux.kallsyms1.S
  LD      .tmp_vmlinux.kallsyms2
arm-linux-gnueabihf-ld: warning: .tmp_vmlinux.kallsyms1.o: missing .note.GNU-stack section implies executable stack
arm-linux-gnueabihf-ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
  KSYMS   .tmp_vmlinux.kallsyms2.S
  AS      .tmp_vmlinux.kallsyms2.S
  LD      vmlinux
arm-linux-gnueabihf-ld: warning: .tmp_vmlinux.kallsyms2.o: missing .note.GNU-stack section implies executable stack
arm-linux-gnueabihf-ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
  SORTTAB vmlinux
  SYSMAP  System.map
  OBJCOPY arch/arm/boot/Image
  MODPOST modules-only.symvers
  GEN     Module.symvers
  Kernel: arch/arm/boot/Image is ready
  GZIP    arch/arm/boot/compressed/piggy_data
  AS      arch/arm/boot/compressed/piggy.o
  LD      arch/arm/boot/compressed/vmlinux
arm-linux-gnueabihf-ld: warning: arch/arm/boot/compressed/bswapsdi2.o: missing .note.GNU-stack section implies executable stack
arm-linux-gnueabihf-ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
arm-linux-gnueabihf-ld: warning: arch/arm/boot/compressed/vmlinux has a LOAD segment with RWX permissions
  OBJCOPY arch/arm/boot/zImage
  Kernel: arch/arm/boot/zImage is ready
arm-linux-gnueabihf-ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
  SORTTAB vmlinux
  SYSMAP  System.map
  OBJCOPY arch/arm/boot/Image
  MODPOST modules-only.symvers
  GEN     Module.symvers
  Kernel: arch/arm/boot/Image is ready
  GZIP    arch/arm/boot/compressed/piggy_data
  AS      arch/arm/boot/compressed/piggy.o
  LD      arch/arm/boot/compressed/vmlinux
arm-linux-gnueabihf-ld: warning: arch/arm/boot/compressed/bswapsdi2.o: missing .note.GNU-stack section implies executable stack
arm-linux-gnueabihf-ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
arm-linux-gnueabihf-ld: warning: arch/arm/boot/compressed/vmlinux has a LOAD segment with RWX permissions
  OBJCOPY arch/arm/boot/zImage
  Kernel: arch/arm/boot/zImage is ready
```

Hình 8. Phần cuối log build kernel

```
  Kernel: arch/arm/boot/zImage is ready
letienthanh@letienthanh-XiaoXinPro-14-AHP9:~/embedded_lab1/kernel/linux-5.15$ cd ~/embedded_lab1/kernel/linux-5.15

ls -lh arch/arm/boot/zImage
file arch/arm/boot/zImage

ls -lh arch/arm/boot/dts/vexpress-v2p-ca9.dtb
file arch/arm/boot/dts/vexpress-v2p-ca9.dtb
-rwxrwxr-x 1 letienthanh letienthanh 4,8M Thg 10 20 12:28 arch/arm/boot/zImage
arch/arm/boot/zImage: Linux kernel ARM boot executable zImage (little-endian)
-rw-rw-r-- 1 letienthanh letienthanh 14K Thg 9  13 10:09 arch/arm/boot/dts/vexpress-v2p-ca9.dtb
arch/arm/boot/dts/vexpress-v2p-ca9.dtb: Device Tree Blob version 17, size=14081, boot CPU=0, string block size=917, DT structure block size=13108
```

Hình 9. ls -lh arch/arm/boot/zImage và file arch/arm/boot/zImage

Phần B. U-Boot

Phần này dùng U-Boot 2022.04 chạy trên QEMU vexpress-a9. Em ghi lại kết quả `printenv`, việc đặt `bootargs` và thông báo kernel panic khi chưa có rootfs.

```
=>
=> version
U-Boot 2022.04 (Sep 13 2026 - 11:59:44 +0700)

arm-linux-gnueabihf-gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
GNU ld (GNU Binutils for Ubuntu) 2.42
=> bdinfo
boot_params = 0x60002000
DRAM bank   = 0x00000000
-> start    = 0x60000000
-> size     = 0x20000000
DRAM bank   = 0x00000001
-> start    = 0x80000000
-> size     = 0x00000004
flashstart  = 0x40000000
flashsize   = 0x04000000
flashoffset = 0x00000000
baudrate    = 38400 bps
relocaddr   = 0x7ff6b000
reloc off   = 0x1f76b000
Build       = 32-bit
current eth = ethernet@3,02000000
ethaddr     = 52:54:00:12:34:56
IP addr     = <NULL>
fdt_blob    = 0x7ffdfbb0
new_fdt     = 0x00000000
fdt_size    = 0x00000000
lmb_dump_all:
 memory.cnt  = 0x1
 memory[0]      [0x60000000-0x80000003], 0x20000004 bytes flags: 0
 reserved.cnt  = 0x2
 reserved[0]    [0x4c000000-0x4c7fffff], 0x00800000 bytes flags: 4
 reserved[1]    [0x7fb26c90-0x7fffffff], 0x004d9370 bytes flags: 0
devicetree  = embed
arch_number = 0x000008e0
TLB addr    = 0x7fff0000
irq_sp      = 0x7fb2aec0
sp start    = 0x7fb2aeb0
Early malloc usage: 370 / 400
=> printenv
arch=arm
baudrate=38400
board=vexpress
```

Hình 10. U-Boot shell: version, bdinfo, printenv

```
=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M'
=>  printenv bootargs
bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M
=>
```

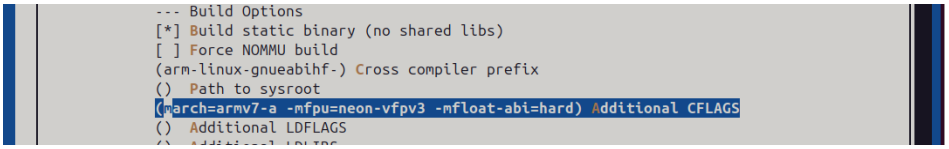
Hình 11. setenv bootargs và printenv bootargs

```
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
sii902x 0-0039: supply iovcc not found, using dummy regulator
sii902x 0-0039: supply cvcc12 not found, using dummy regulator
i2c i2c-0: Added multiplexed i2c bus 2
drm-clcd-pl111 1001f000.clcd: assigned reserved memory node vram@4c000000
drm-clcd-pl111 1001f000.clcd: using device-specific reserved memory
drm-clcd-pl111 1001f000.clcd: core tile graphics present
drm-clcd-pl111 1001f000.clcd: this device will be deactivated
drm-clcd-pl111 1001f000.clcd: Versatile Express init failed - -19
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: found bridge on endpoint 0
drm-clcd-pl111 10020000.clcd: Using non-panel bridge
[drm] Initialized pl111 1.0.0 20170317 for 10020000.clcd on minor 0
Console: switching to colour frame buffer device 128x48
drm-clcd-pl111 10020000.clcd: [drm] fb0: pl111 frame buffer device
ALSA device list:
  #0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
input: ImExPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/bus@40000000:motherboard-bus@40000000:iofpga@7,
00000000/10007000.kmi/serio1/input/input2
VFS: Cannot open root device "ram0" or unknown-block(0,0): error -6
Please append a correct "root=" boot option; here are the available partitions:
1f00          131072 mtdblock0
 (driver?)
1f01           32768 mtdblock1
 (driver?)
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.15.0 #1
Hardware name: ARM-Versatile Express
[<8010f4c4>] (unwind_backtrace) from [<8010b2dc>] (show_stack+0x10/0x14)
[<8010b2dc>] (show_stack) from [<8084b4ac>] (dump_stack_lvl+0x40/0x4c)
[<8084b4ac>] (dump_stack_lvl) from [<808473b0>] (panic+0xfc/0x2e8)
[<808473b0>] (panic) from [<80b01748>] (mount_block_root+0x1d0/0x274)
[<80b01748>] (mount_block_root) from [<80b01bd8>] (prepare_namespace+0x150/0x18c)
[<80b01bd8>] (prepare_namespace) from [<8084fcbc>] (kernel_init+0x10/0x124)
[<8084fcbc>] (kernel_init) from [<80100130>] (ret_from_fork+0x14/0x24)
Exception stack(0x810a7fb0 to 0x810a7ff8)
7fa0:                                     00000000 00000000 00000000 00000000
7fc0: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
7fe0: 00000000 00000000 00000000 00000000 00000013 00000000
---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0) ]---
```

Hình 12. Kernel panic khi chưa có initramfs

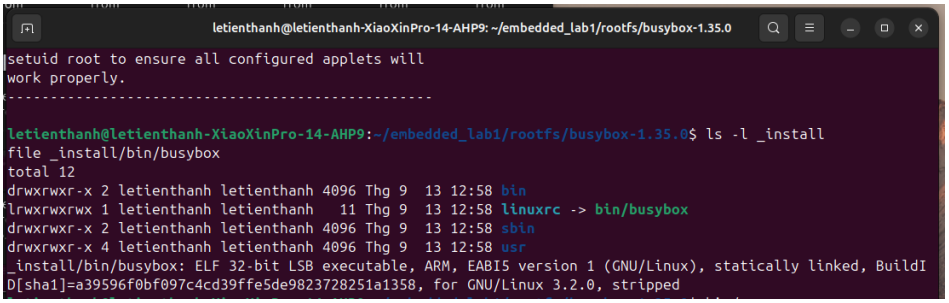
Phần C. BusyBox, initramfs và boot

BusyBox 1.35.0 được cấu hình trong `menuconfig` với Build static binary bật [*], Cross compiler prefix là `arm-linux-gnueabihf-` và Additional CFLAGS là `-march=armv7-a -mfpu=neon-vfpv3 -mfloat-abi=hard`.



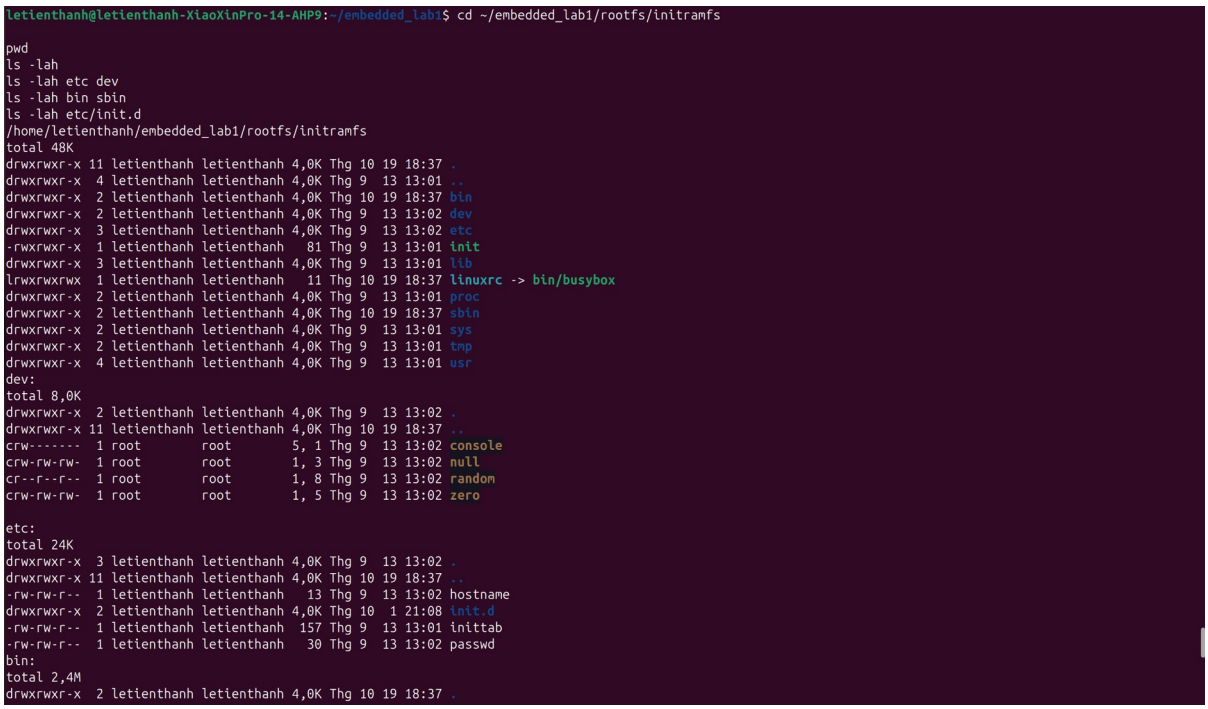
Hình 13. BusyBox Settings: static, cross compiler prefix, CFLAGS

Sau `make` và `make install`, `file _install/bin/busybox` báo ELF 32-bit LSB executable, ARM, EABI5, statically linked, stripped. Thư mục `_install` có `bin`, `sbin`, `usr` và `linuxrc`.



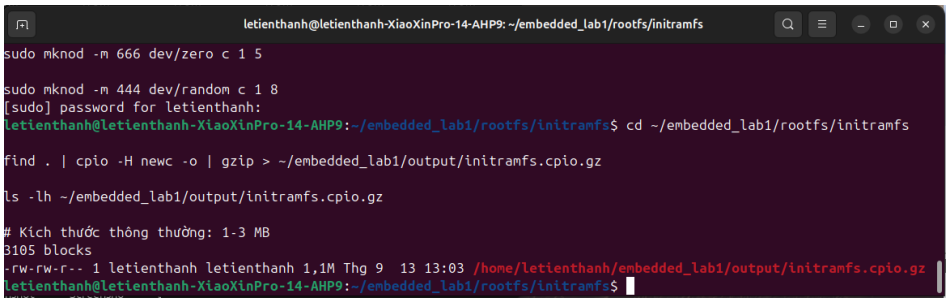
Hình 14. ls -l _install và file _install/bin/busybox

Cấu trúc rootfs nằm trong `rootfs/initramfs`.



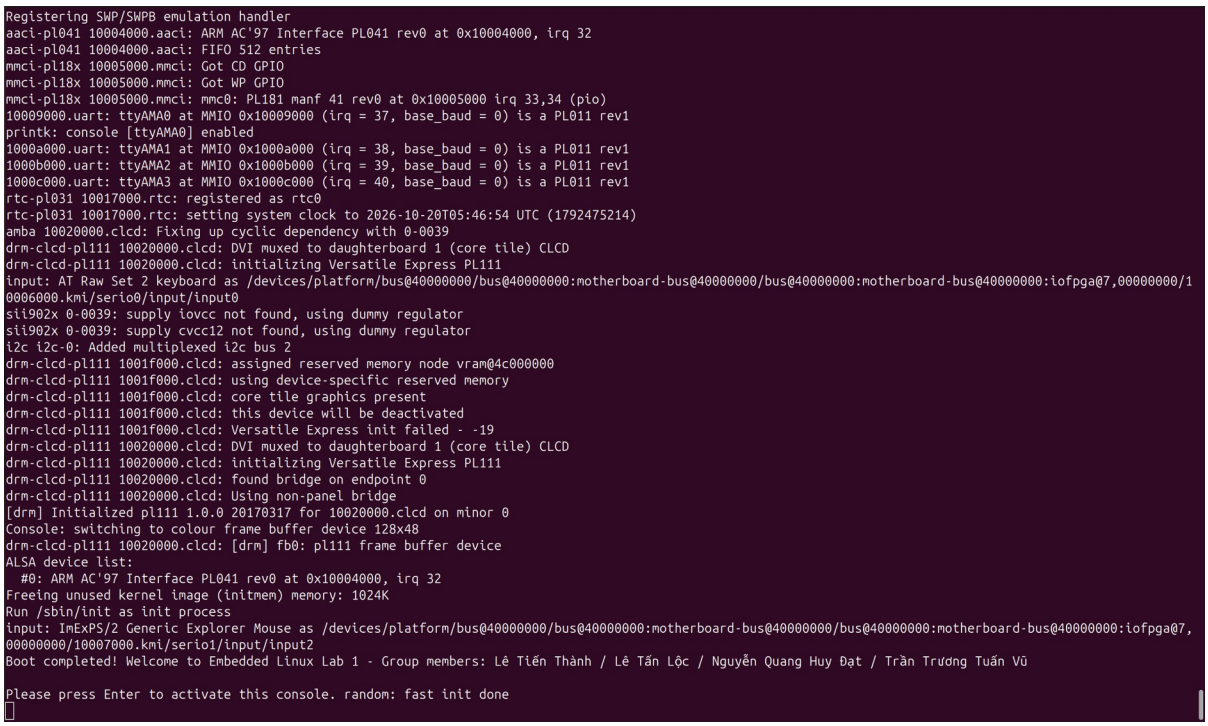
Hình 15. Nội dung thư mục rootfs/initramfs

Em tạo các device node bằng `mknod` (`null`, `console`, `zero`, `random`), rồi đóng gói bằng `find . | cpio -H newc -o | gzip`. File `initramfs.cpio.gz` nặng 1,1 MB, nằm trong khoảng 1 đến 3 MB mà đề nêu.



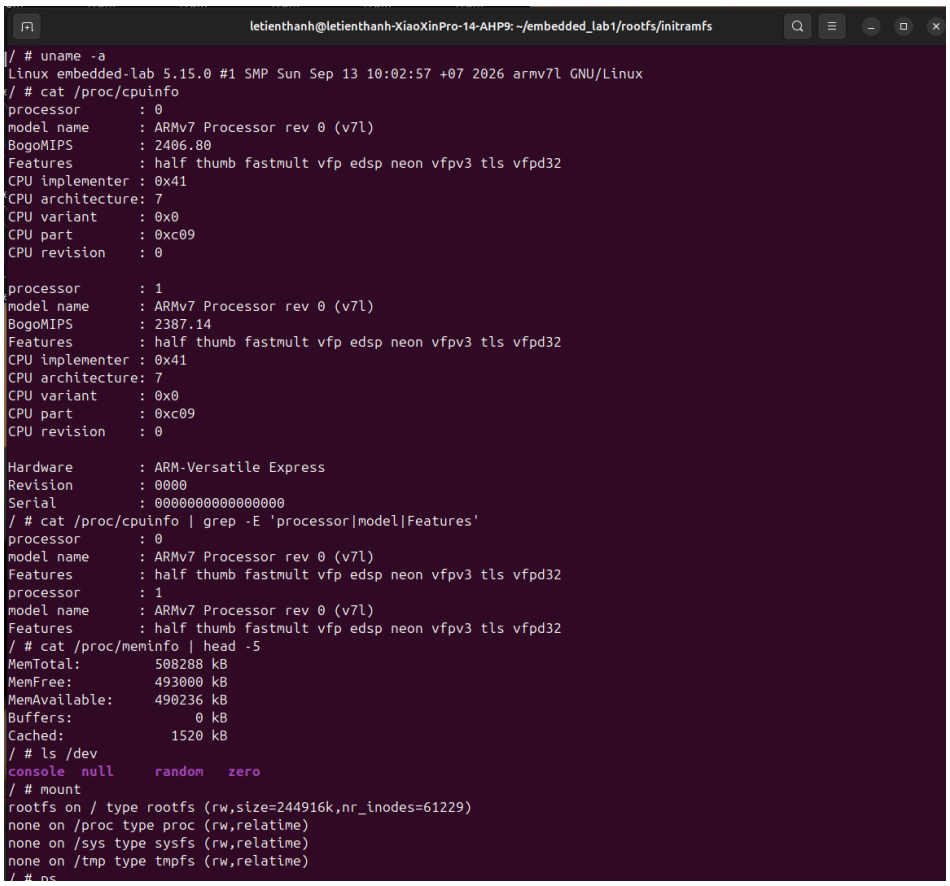
Hình 16. Tạo device node và đóng gói initramfs.cpio.gz

Lệnh boot hoàn chỉnh trên QEMU vexpress-a9 dùng zImage, DTB và initramfs ở trên.



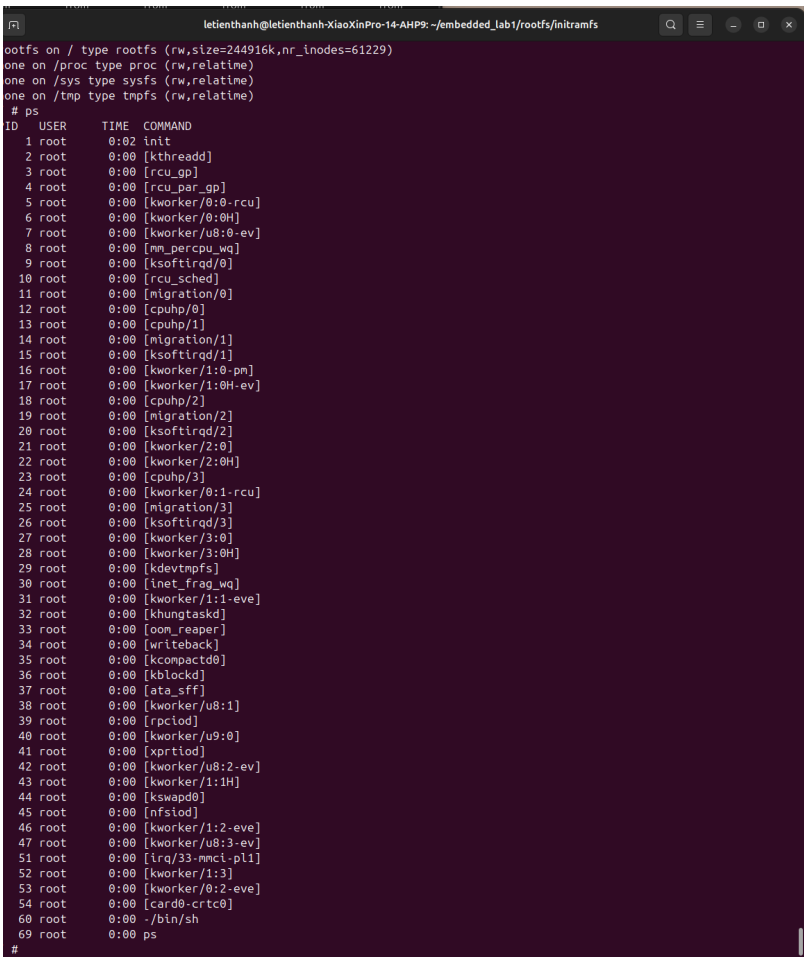
Hình 17. QEMU khởi động Linux thành công và vào BusyBox shell

Sau khi boot lên shell, em chạy các lệnh kiểm tra. `uname -a` cho `Linux embedded-lab 5.15.0 #1 SMP Sun Sep 13 10:02:57 +07 2026 armv7l GNU/Linux`. Tên máy `embedded-lab` do `rcS` đặt, và chữ SMP trong chuỗi khẳng định kernel được build với SMP. `/proc/cpuinfo` liệt kê hai processor, đều là ARMv7 Processor rev 0 (v7l) với Features `half thumb fastmult vfp edsp neon vfpv3 tls vfpd32`, CPU implementer 0x41, CPU architecture 7, CPU variant 0x0, CPU part 0xc09, CPU revision 0, Hardware là ARM-Versatile Express. `MemTotal` là 508288 kB (khoảng 496 MiB trong 512 MiB cấp cho máy ảo, phần còn lại kernel giữ lại). `/dev` có bốn node là `console`, `null`, `random`, `zero`, đúng bốn node em tạo bằng `mknod`. Lệnh `mount` cho thấy `rootfs` trên `/`, cùng `proc`, `sysfs` và `tmpfs` trên `/tmp`.



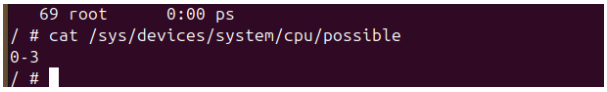
Hình 18. uname -a, /proc/cpuinfo, /proc/meminfo, ls /dev, mount trong BusyBox shell

Lệnh `ps` cho thấy `init` là PID 1, các kernel thread, và `-/bin/sh` là PID 60. Danh sách có các thread theo từng CPU như `cpuhp/N`, `migration/N`, `ksoftirqd/N`.



Hình 19. ps trong BusyBox shell

Lệnh `cat /sys/devices/system/cpu/possible` trả về `0-3`. Đề mong đợi `0-1` khi chạy `-smp 2`, mục 5 giải thích khác biệt này.



Hình 20. cat /sys/devices/system/cpu/possible

Mục 4. Phân tích và trả lời câu hỏi

Câu 1. gnueabi và gnueabihf

Hai toolchain khác nhau ở quy ước gọi hàm (ABI) với số thực. arm-linux-gnueabi là soft-float ABI: số thực được truyền qua thanh ghi số nguyên r0 đến r3, và nếu không có FPU thì phép tính chạy bằng hàm thư viện. arm-linux-gnueabihf là hard-float ABI: số thực truyền thẳng qua thanh ghi của FPU (VFP) và phép tính dùng lệnh VFP hoặc NEON. Cortex-A9 trong lab có FPU, /proc/cpuinfo ở Hình 18 liệt kê vfp, neon, vfpv3, vfpd32. gnueabihf dùng hard-float calling convention và phù hợp với userspace ARM hard-float. gnueabi dùng quy ước truyền đối số soft-float; việc mã có sử dụng lệnh VFP hay không còn phụ thuộc tùy chọn biên dịch (ví dụ softfp), không thể kết luận gnueabi luôn bỏ phí FPU.

Hai ABI không trộn lẫn được. Binary và thư viện mỗi bên ghi một thẻ ABI khác nhau (`Tag_ABI_VFP_args`), trình liên kết từ chối ghép, và trình nạp động cũng có tên khác (`ld-linux.so.3` cho gnueabi, `ld-linux-armhf.so.3` cho gnueabihf). Với kernel thì chuyện nhẹ

hơn: kernel luôn biên dịch ở chế độ soft-float và không dùng FPU trong kernel mode, nên kernel build bằng gnueabi vẫn có thể boot trên A9. Vấn đề thật xuất hiện ở userspace, ví dụ BusyBox build bằng toolchain này rồi nạp cùng thư viện của toolchain kia. Phần kernel này em suy ra từ kiến trúc, em chưa tự build kernel bằng gnueabi để thử.

Câu 2. Memory map bắt đầu từ 0x60000000

Trên Versatile Express, RAM của tile Cortex-A9 được ánh xạ từ 0x60000000. Vùng địa chỉ thấp hơn dành cho thứ khác: flash NOR của bo mạch, các thiết bị ngoại vi và vùng ánh xạ lại để CPU lấy lệnh đầu tiên sau reset (reset vector ở 0x0). Versatile PB có RAM ở 0x00000000 nên không có sự khác biệt này.

Hệ quả với U-Boot là mọi địa chỉ nạp phải nằm trong cửa sổ RAM bắt đầu từ 0x60000000. Đề dùng bootz 0x60100000 0x64000000 0x60000000, nghĩa là zImage ở 0x60100000, initramfs ở 0x64000000, DTB ở 0x60000000, ba địa chỉ đều trong dải RAM theo đề. Tuy vậy, vẫn phải kiểm tra vùng thực tế U-Boot chiếm dụng, không chỉ kiểm tra địa chỉ nằm trong RAM. Nếu em chép cách đặt của board có RAM ở 0x0 (ví dụ nạp kernel ở 0x00100000) thì U-Boot ghi vào vùng không phải RAM nên kernel không nằm ở chỗ nó sẽ chạy. Nếu địa chỉ nằm trong RAM nhưng hai vùng chồng nhau, ví dụ initramfs đè lên zImage hoặc DTB, thì sẽ gặp các lỗi như sai magic của zImage, không tìm thấy DTB hợp lệ hoặc treo ngay sau dòng "Starting kernel". Em chưa thử đặt sai địa chỉ để xem lỗi thật, các lỗi trên là những gì em dự đoán từ cách bootz hoạt động.

Câu 3. MIDR 0x410fc090

MIDR (Main ID Register) có 32 bit và 0x410fc090 tách ra như sau. Bit 31 đến 24 là implementer = 0x41, tức ARM Ltd. (mã ASCII "A"). Bit 23 đến 20 là variant = 0x0. Bit 19 đến 16 là architecture = 0xF, nghĩa là kiến trúc được xác định qua các thanh ghi ID khác (CPUID scheme), nên kernel in ra "ARMv7". Bit 15 đến 4 là part number = 0xC09. Bit 3 đến 0 là revision = 0, tức bản r0p0.

Part number 0xC09 tương ứng Cortex-A9. Kết quả này khớp với `/proc/cpuinfo` của em ở Hình 18: CPU implementer 0x41, CPU architecture 7, CPU variant 0x0, CPU part 0xc09, CPU revision 0.

Câu 4. Static linking cho BusyBox

Initramfs của em chỉ có BusyBox và vài file cấu hình, không có thư viện nào. Một BusyBox liên kết động cần trình nạp động và libc ngay khi kernel exec nó làm PID 1. Nếu thiếu, kernel không chạy được `init` và dừng với lỗi kiểu "Attempted to kill init". Build static (Hình 13 và 14: statically linked) gói hết mã thư viện vào một file, nên rootfs nhỏ và không phụ thuộc gì.

Nếu build động cho ARMv7 hard-float, cần chép ít nhất ba file: `ld-linux-armhf.so.3` (trình nạp động), `libc.so.6` và `libm.so.6`. BusyBox thường còn cần `libresolv.so.2`. Chúng nằm trong sysroot của toolchain, ở `/usr/arm-linux-gnueabihf/lib/` trên máy em, và phải chép vào `lib/arm-linux-gnueabihf` trong initramfs (thư mục này đề đã yêu cầu tạo ở bước C.2). Danh sách chính xác kiểm bằng `arm-linux-gnueabihf-readelf -d busybox | grep NEEDED` trên bản build động. Em chưa build bản động nên chưa chạy lệnh này.

Câu 5. SMP với -smp 2

Khi bật SMP, kernel phải làm thêm nhiều việc so với uniprocessor. CPU 0 phải đánh thức CPU phụ qua cơ chế riêng của nền tảng (phụ thuộc cơ chế khởi động CPU phụ của nền tảng, SCU và cấu hình firmware/device tree) rồi CPU phụ tự chạy phần khởi tạo của nó. Kernel dựng dữ liệu theo từng CPU (per-CPU data), kernel thread theo từng CPU, và bộ lập lịch có thêm cân bằng tải giữa các CPU. Các CPU giao tiếp với nhau bằng ngắt liên CPU (IPI) qua GIC, các khóa phải thật sự khóa và cache phải nhất quán giữa các lõi.

Trong hệ thống của em có thể thấy hậu quả ngay: `/proc/cpuinfo` liệt kê hai processor với BogoMIPS riêng (2406,80 và 2387,14), và `ps` có các thread theo từng CPU như `migration/1`, `ksoftirqd/1`, `kworker/1:0`.

Phần so sánh boot log giữa `-smp 1` và `-smp 2` cần hai lần chạy QEMU. Với `-smp 2`, log có thêm các dòng bring-up CPU phụ như `CPU1: thread -1, cpu 1, socket 0, mpidr 80000001` (CPU 1 đã được bật), `smp: Brought up 1 node, 2 CPUs` và `SMP: Total of 2 processors activated` (kernel đã đếm đủ số CPU và cộng BogoMIPS). Với `-smp 1` các dòng này không xuất hiện hoặc chỉ ghi một CPU. Hai hình dưới là bằng chứng từ chính hai lần chạy của em.

```
Boot completed! Welcome to Embedded Linux Lab 1 - Group members: Lê Tiến Thành / Lê Tấn Lộc / Nguyễn Quang Huy Đạt / Trần Trương Tuấn Vũ

Please press Enter to activate this console. random: fast init done

/ # cat /sys/devices/system/cpu/online
0
/ # dmesg | grep -E "smp:|SMP:|CPU0:|CPU1:"
CPU0: Spectre v2: using BPIALL workaround
CPU0: thread -1, cpu 0, socket 0, mpidr 80000000
smp: Bringing up secondary CPUs ...
smp: Brought up 1 node, 1 CPU
SMP: Total of 1 processors activated (683.62 BogoMIPS).
/ #
```

Hình 21. Boot log của -smp 1: dmesg | grep -iE "smp|cpu"

```
input: ImExPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/bus@40000000:motherboard-bus@40000000:iofpga@7,
00000000/10007000.kmi/serio1/input/input2
Boot completed! Welcome to Embedded Linux Lab 1 - Group members: Lê Tiến Thành / Lê Tấn Lộc / Nguyễn Quang Huy Đạt / Trần Trương Tuấn Vũ

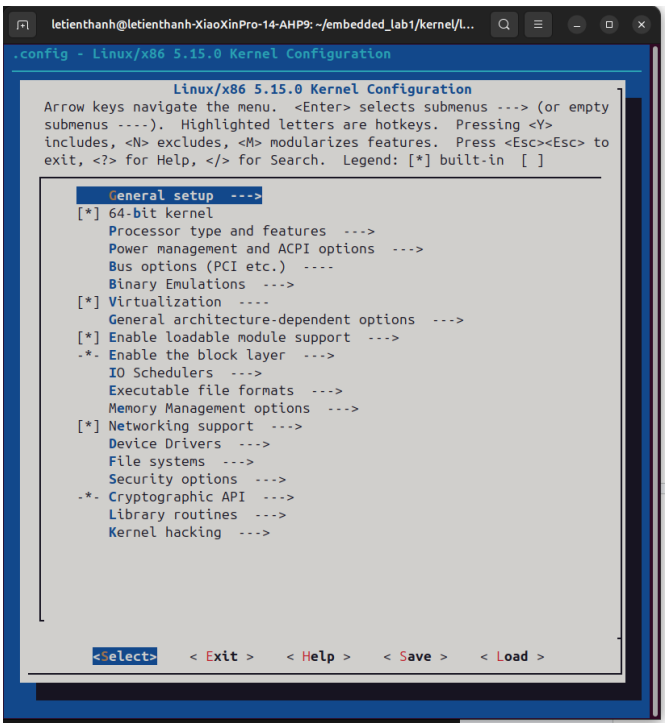
Please press Enter to activate this console. random: fast init done
echo "===== LAB 1 BOOT SUCCESS ====="
uname -a
cat /sys/devices/system/cpu/online
ps/ # uname -a
Linux embedded-lab 5.15.0 #1 SMP Sun Sep 13 10:02:57 +07 2026 armv7l GNU/Linux
/ # cat /sys/devices/system/cpu/online
0-1
/ # psecho "===== SMP 2 CPUs ====="
-/bin/sh: psecho: not found
/ # dmesg | grep -iE "smp|cpu"
Booting Linux on physical CPU 0x0
Linux version 5.15.0 (letienthanh@letienthanh-XiaoXinPro-14-AHP9) (arm-linux-gnueabihf-gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, GNU ld (GNU Binutils for U
buntu) 2.42) #1 SMP Sun Sep 13 10:02:57 +07 2026
CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
CPU: PIPT / VIPT nonaliasing data cache, VIPT nonaliasing instruction cache
percpu: Embedded 15 pages/cpu s30156 r8192 d23092 u61440
pcpu-alloc: s30156 r8192 d23092 u61440 alloc=15*4096
pcpu-alloc: [0] 0 [0] 1 [0] 2 [0] 3
printk: log_buf_len individual max cpu contribution: 4096 bytes
printk: log_buf_len total cpu_extra contributions: 12288 bytes
SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=4, Nodes=1
rcu:    RCU restricting CPUs from NR_CPUS=8 to nr_cpu_ids=4.
rcu: Adjusting geometry for rcu_fanout_leaf=16, nr_cpu_ids=4
smp_twd: clock not found -2
CPU: Testing write buffer coherency: ok
CPU0: Spectre v2: using BPIALL workaround
CPU0: thread -1, cpu 0, socket 0, mpidr 80000000
smp: Bringing up secondary CPUs ...
CPU1: thread -1, cpu 1, socket 0, mpidr 80000001
CPU1: Spectre v2: using BPIALL workaround
CPU2: failed to boot: -38
CPU3: failed to boot: -38
smp: Brought up 1 node, 2 CPUs
SMP: Total of 2 processors activated (1421.72 BogoMIPS).
CPU: All CPU(s) started in SVC mode.
cpuidle: using governor ladder
ledtrig-cpu: registered to indicate activity on CPUs
/ #
```

Hình 22. Boot log của -smp 2: dmesg | grep -iE "smp|cpu"

Mục 5. Nhận xét và khó khăn gặp phải

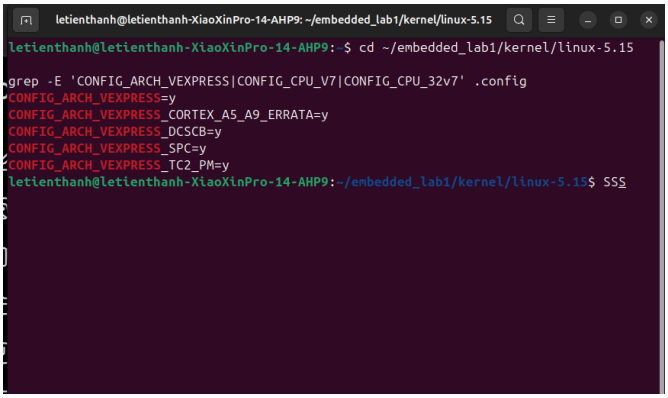
Em gặp ba chuyện đáng ghi lại.

Thứ nhất, lần đầu em chạy `make menuconfig` mà chưa đặt `ARCH=arm`. Tiêu đề của menu là "Linux/x86 5.15.0 Kernel Configuration" và có dòng `64-bit kernel`, nghĩa là em đang cấu hình kernel cho máy x86 chứ không phải ARM. Cấu hình đó không dùng được cho vexpress-a9. Em thoát ra, đặt `export ARCH=arm` và `export CROSS_COMPILE=arm-linux-gnueabihf-`, rồi chạy lại. Lần sau menu có tiêu đề "Linux/arm 5.15.0 Kernel Configuration" và có mục System Type (Hình 3, 5 và 6).

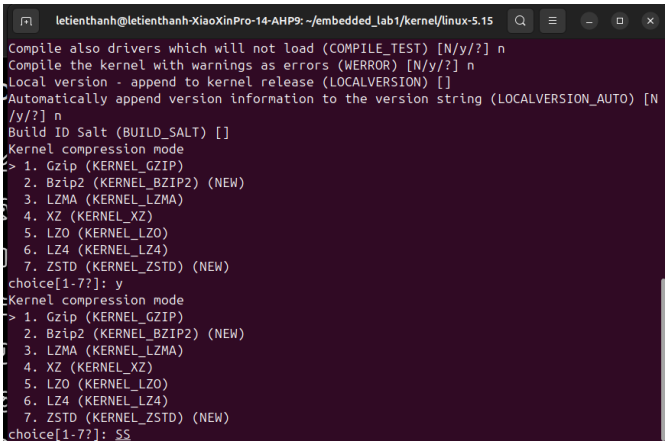


Hình 23. menuconfig chạy sai kiến trúc (Linux/x86, có 64-bit kernel)

Thứ hai, sau khi chuyển kiến trúc, `grep` kiểm tra `.config` chưa thấy `CONFIG_CPU_V7` và `CONFIG_CPU_32v7` (chỉ có các tùy chọn con của `ARCH_VEXPRESS`). Em chạy lại cấu hình với `ARCH=arm` (hệ thống hỏi thêm các tùy chọn mới, ví dụ kiểu nén kernel) rồi chạy `make vexpress_defconfig`. Sau đó `grep` đã thấy `CONFIG_CPU_V7=y` và `CONFIG_CPU_32v7=y` (kết quả ở Hình 2). Bài học là phải kiểm `.config` bằng `grep` trước khi build, vì build với `.config` sai sẽ mất 10 đến 20 phút mới biết.



Hình 24. grep trước khi sửa: chưa có CONFIG_CPU_V7



Hình 25. Cấu hình lại với ARCH=arm: các câu hỏi cho tùy chọn mới

Thứ ba là một điểm khác với mong đợi của đề: `/sys/devices/system/cpu/possible` trả về `0-3` trong khi đề nói `0-1` với `-smp 2`, và `/proc/cpuinfo` chỉ liệt kê hai processor (Hình 18, 20). Đây không phải lỗi. Em hiểu `possible` là số CPU mà kernel có thể quản lý, lấy từ device tree của vexpress-v2p-ca9 (khai báo bốn lõi), còn số CPU thật sự chạy do tham số `-smp` của QEMU quyết định. Điều em chưa giải thích được là `ps` vẫn có thread như `cpuhp/2`, `cpuhp/3` và `kworker/3:0` cho các CPU 2 và 3. Để biết chính xác, em cần xem thêm `/sys/devices/system/cpu/online` và `present`, em chưa làm.

Mục 6. Kiểm thử bổ sung trên bản tải từ GitHub (21/10/2026)

Đợt kiểm thử bổ sung được thực hiện sau khi tải repository LOS201 từ GitHub về thư mục riêng LOS201_LAB1_TEST. Mục đích là xác nhận các tệp đã đưa lên kho mã nguồn có thể khởi động độc lập, không chỉ chạy được tại thư mục phát triển ban đầu.

6.1. Kiểm tra hiện vật và khả năng tái lập

Repository được kiểm tra tại nhánh main, commit 95725ae (ADD Assignment 01 02), có working tree sạch và Lab 1 không phải submodule. Các tệp output/zImage, output/vexpress-v2p-ca9.dtb, output/initramfs.cpio.gz, configs/kernel.config và configs/busybox.config đều tồn tại, có dữ liệu.

Lệnh file xác nhận zImage là Linux kernel ARM boot executable, U-Boot là ELF 32-bit ARM EABI5, BusyBox là ELF 32-bit ARM EABI5 statically linked. Kernel config có CONFIG_ARCH_VEXPRESS=y, CONFIG_BLK_DEV_INITRD=y, CONFIG_DEVTMPFS=y và CONFIG_SERIAL_AMBA_PL011=y.

6.2. Boot trực tiếp Kernel và initramfs

```
qemu-system-arm -M vexpress-a9 -m 512M \
  -kernel output/zImage \
  -dtb output/vexpress-v2p-ca9.dtb \
  -initrd output/initramfs.cpio.gz \
  -append "console=ttyAMA0 rdinit=/sbin/init" -nographic
```

Kết quả: Linux 5.15.0 ARMv7 khởi động đến BusyBox shell; /sbin/init có PID 1; các filesystem proc, sysfs và tmpfs trên /tmp được mount; /proc/cmdline hiển thị đúng chuỗi console=ttyAMA0 rdinit=/sbin/init. Các lệnh uname -a, mount, ps, ls /bin được thực thi thành công.

6.3. Boot chuỗi U-Boot → Kernel → BusyBox

U-Boot 2022.04 hiển thị DRAM: 512 MiB và vào dấu nhắc =>. Các lệnh printenv, setenv và help bootz hoạt động. Lần thử với DTB ở 0x60000000 gặp FDT_ERR_BADSTRUCTURE; sau khi chuyển DTB sang 0x66000000, lệnh fdt header trả magic 0xd00dfeed, totalsize 0x3701 và fdt print /chosen cho thấy node chosen hợp lệ.

```
bootargs=console=ttyAMA0,115200 rdinit=/sbin/init
bootcmd=bootz 0x62000000 0x64000000:0x1a0000 0x66000000
```

```
Kernel image @ 0x62000000
Loading Ramdisk ... OK
Loading Device Tree ... OK
Starting kernel ...
Unpacking initramfs...
Run /sbin/init as init process
Boot completed! Welcome to Embedded Linux Lab 1 ...
/ #
```

Sau khi vào shell, uname -a cho Linux embedded-lab 5.15.0 armv7l; cat /proc/cmdline trả về chuỗi bootargs ở trên; ps xác nhận PID 1 là init. Như vậy chuỗi khởi động đầy đủ qua U-Boot đã được kiểm chứng bằng log thực tế của bản clone.

Lưu ý kỹ thuật: tham số 0x1a0000 là kích thước raw-initrd đã sử dụng trong lần chạy có log; báo cáo hiện chưa kèm kết quả stat xác nhận kích thước byte chính xác của output/initramfs.cpio.gz. Cần tính lại và dùng đúng kích thước thật nếu tái chạy test. Địa chỉ RAM nạp Kernel/DTB của phép thử thành công khác ví dụ minh họa trong đề; chúng được chọn để tránh vùng dữ liệu xung đột khi boot.